

Stellaroid.tech Audit — SEO, Performance & Roadmap

Summary

Stellaroid Earn is a polished web dApp that demonstrates on-chain credential verification and payment on the Stellar testnet. The site anchors certificate hashes to a Soroban contract and allows approved issuers to register/verify credentials while employers pay graduates directly. It uses Next.js 15 with React 19, Tailwind CSS and client-side state for wallet management. Meta tags, canonical links and structured data are already strong; a cut-over runbook ensures `www.stellaroid.tech` and `earn.stellaroid.tech` permanently redirect to the apex domain ¹. However, heavy client components (navigation, animated backgrounds, activity snackbars) and Framer Motion animations add significant JavaScript weight and can harm Core Web Vitals. This audit identifies current strengths, issues and a phased roadmap to harden SEO, cut down client code and improve performance.

Current strengths

- **Robust SEO foundations** – Each page exports metadata using a helper; the landing page sets title, description and keywords ¹. The `<head>` includes canonical and hreflang links (English and Tagalog) and uses Open Graph/Twitter tags with preview images ¹. Proof pages are server components that fetch on-chain data and output JSON-LD for breadcrumb, digital document and article schemas ¹.
- **Canonical domain & redirects** – The cut-over runbook confirms `www` and `earn` subdomains redirect via `308` to `https://stellaroid.tech` ¹. A status route (`/status`) and health API allow uptime checks and DNS verification.
- **Accessible layout & localised content** – The nav includes a “Skip to content” link and an accessible mobile drawer. The `LocalizedHero` component reads locale from a context and renders Tagalog/English copy. A locale toggle persists preferences to `localStorage` and cookies.
- **Server-side proof pages** – The `/proof/[hash]` route fetches certificate and issuer records server-side and caches the response for 60 s. It produces JSON-LD for breadcrumb list, digital document and article schema, ensuring search engines can index each credential as a standalone article.
- **Cut-over documentation** – The project includes a detailed runbook (`STELLAROID_TECH_CUTOVER.md`) and roadmap, ensuring domain redirects and fallback checks remain consistent ¹. Environment variables are enumerated in `README.md` for transparency.

Issues & opportunities

Area	Findings & impact
Navigation & layout	The <code>SiteNav</code> component is a client component that imports <code>useState</code> and <code>usePathname</code> to manage mobile menus and highlight active links. This means the entire nav is hydrated on every page. The mobile drawer toggles state and loads icons; even minor nav changes require JS.
Animated backgrounds	The <code>HeroOrbs</code> component imports <code>framer-motion</code> and animates two orbs across the screen. Framer Motion adds ~20-30 KB of JS and is loaded for all visitors even though the animations are purely decorative.
Activity snackbar	<code>ActivitySnackBar</code> attaches scroll listeners and uses <code>requestAnimationFrame</code> to drive a progress spinner and auto-hide timer. It shows on-chain activity after the user scrolls beyond a threshold. The scroll listeners and animation frames run even when no activity exists, increasing INP and CPU usage.
Client-side wallet & forms	The <code>/app</code> route wraps the entire experience in <code>FreighterWalletProvider</code> , <code>AppShell</code> , <code>NextActionCard</code> , <code>RegisterForm</code> , <code>VerifyForm</code> , <code>PayForm</code> and others. <code>AppExperience</code> uses multiple <code>useState</code> hooks, refs and <code>useEffect</code> to manage scroll behaviour and step progression. This heavy client flow is necessary for interactive testnet transactions, but it means 100+ KB of JS and complex hydration.
Locale persistence	<code>LocaleToggle</code> stores the chosen language in <code>localStorage</code> and dispatches a custom event. On first paint the server may render one locale while the client hydrates to another, causing a content shift.
Lack of sitemap & robots	There is no automatically generated <code>sitemap.xml</code> or <code>robots.txt</code> in the code. Important pages (proofs, app, issuer, status, metrics, slides) are not listed for crawlers.
Performance budgets & analytics	The site imports <code>@vercel/analytics</code> and includes animated hero backgrounds and numerous icons. There is no explicit performance budget or measurement in the repository besides manual Playwright tests. Heavy pages like <code>/app</code> might exceed Core Web Vitals thresholds on slower devices.

Recommended roadmap

Focus on one main quest at a time: secure SEO, then reduce hydration, then optimize heavy flows. Resist adding features until fundamentals are solid.

Phase 1 – SEO & domain integrity

1. **Enforce permanent redirects** – Confirm `www.stellaroid.tech` and `earn.stellaroid.tech` return `308` to the apex domain by adding them to the Vercel project with “redirect to root” configuration. Keep the canonical link set to `https://stellaroid.tech` ¹.

2. **Unique titles & descriptions** – For each route (`/about` , `/issuer` , `/employer` , `/metrics` , `/slides`) export a unique title and meta description using `buildPageMetadata` . Avoid generic copy; highlight the page's purpose and include relevant keywords.
3. **Language & direction attributes** – Add `lang="en"` and `lang="tl"` attributes to `<html>` based on the `LocaleToggle` cookie. Provide a server-side fallback by reading `stellaroid:locale` from cookies in `_middleware` or using Next Internationalization API.
4. **Generate a sitemap & robots.txt** – Create an API route (e.g., `/api/sitemap`) to generate a `sitemap.xml` with URLs for the landing page, about, issuer, employer, app (static), status, metrics, slides and proof hashes. Use `revalidate` on the proof sitemap segment to add new credentials. Add a `robots.txt` that allows crawling but disallows `/api` and `/_next` paths.
5. **Structured data enhancements** – Continue generating JSON-LD for proofs (digital document and article). Add a `BreadcrumbList` and `WebSite` schema on all pages. On the landing page, add an `FAQ` schema summarising the three-step process (register, verify, pay).

Phase 2 – Trim client-side footprint

1. **Convert nav to server component** – Move `SiteNav` to a server component that renders static links. Only the mobile menu toggle needs to be a client island; use `<details><summary>Menu</summary>...</details>` for the mobile nav to avoid `useState` and `usePathname` . This cuts hydration on every page.
2. **Remove Framer Motion from hero** – Replace the `HeroOrbs` component with a CSS animation using `@keyframes` (as done in `HeroBg`) or a static gradient. If subtle movement is desired, wrap the orbs in a dynamic import (`next/dynamic({ ssr: false })`) so the animation code is downloaded only after the page is interactive.
3. **Simplify `ActivitySnackbar`** – Show live activity only on the `/app` route. Remove scroll detection and rAF timers; instead provide a small link under “Recent contract events” to view all events. For dynamic updates, poll serverless function in the background and update UI with a toast component from `sonner` (already a dependency) when new events arrive.
4. **Locale persistence on the server** – Replace `localStorage` usage in `LocaleToggle` with cookies. Set the locale cookie when the user toggles the language and read it in the server `generateMetadata` and `i18n` hook. This removes hydration mismatch.
5. **Lazy-load heavy modules** – Use `next/dynamic` to load `qrcode` , `@stellar/freighter-api` , and large forms only when required. For example, hide the wallet connect and Freighter components behind a dynamic import triggered when the user clicks “Issue certificate” or “Pay student”.

Phase 3 – Optimize the `/app` experience

1. **Server actions & streaming** – Migrate parts of `RegisterForm` , `VerifyForm` and `PayForm` to Next.js server actions. For example, the API calls to `stellar` and transaction building can run on the serverless edge and return results via streaming or `action` s. This reduces client code and protects secret keys.
2. **Use `useFormState` & `useOptimistic`** – Implement forms using React Server Components with `useFormState` to manage state updates. Display optimistic UI while waiting for transaction confirmation. This reduces client reactivity and improves INP.

3. **Hide wallet provider behind user intent** – Only load the `FreighterWalletProvider` when the user visits `/app` and attempts to connect their wallet. Provide a call-to-action that loads the provider dynamically, reducing initial load on `/app`.
4. **Progress rail & step animations** – Simplify milestone rails and step transitions. Avoid smooth scrolling triggered by `useEffect`; instead use anchor links or CSS scroll snapping. Remove heavy `MilestoneRail` if progress can be communicated through text and colour alone.

Phase 4 – Assets & performance budgets

1. **Font & icon optimization** – Use `next/font` to self-host fonts and load only required subsets. Replace `lucide-react` icons with a custom sprite sheet or `svg` components imported individually. This reduces unused icons in the bundle.
2. **Image handling** – Replace `<img>` tags with `<Image>` from `next/image` for hero illustrations and event placeholders. Set appropriate `sizes` and `priority` props to control loading. Combine small static illustrations into a single sprite to reduce HTTP requests.
3. **Preconnect & caching** – Preconnect to `soroban-testnet.stellar.org` and `stellar.expert` in the `<head>` to reduce DNS/connection overhead. Set long-term cache headers for static assets (SVGs, CSS) via Vercel configuration. Use `revalidate` or `cache-control` for server responses.
4. **Analytics & measurement** – Ensure `@vercel/analytics` is loaded after `load` (`defer` attribute). Use Lighthouse CI or Vercel Speed Insights to monitor LCP (largest hero text or image), INP (navigation and button interactions) and CLS (layout shift when loading locale or snackbars). Define budgets (e.g., JS < 150 KB per route) and fail the build when exceeded.

Phase 5 – Continuous improvement & future plans

1. **Status & health checks** – Keep `/status` up to date with contract IDs, environment info and last deployment time. Include an uptime monitor that returns JSON for external health checks. This helps maintain trust with pilot issuers and employers.
2. **Documentation & onboarding** – Consolidate the many docs (`ROADMAP.md`, `MAINTENANCE.md`) into a public `docs` page on the site. Provide a clear “Getting started” flow for issuers and employers with call-to-action buttons.
3. **User feedback loop** – Implement a simple feedback form or use GitHub issues to collect feedback. For the bootcamp pilot, track drop-off points (registration vs verification vs payment) and refine flows accordingly.

Conclusion

Stellaroid Earn showcases an impressive blend of on-chain functionality and thoughtful design. Its strong SEO foundations, canonical URL discipline and robust documentation prove that the team values trust and transparency. However, the current implementation relies heavily on client-side React hooks, animated backgrounds and snackbars, which inflate the JavaScript bundle and can hinder Core Web Vitals. By following the phased roadmap above—first solidifying SEO and domain behaviour, then trimming client hydration, optimizing the `/app` experience and assets, and setting measurable performance budgets—the project can mature from a bootcamp demo into a scalable, credible credential verification product. Execute each phase decisively and avoid unnecessary polish until the fundamentals are in place.

1 Stellaroid Earn — Proof & Payment on Stellar
<https://stellaroid.tech/>